

CS-2009 Design and Analysis of Algorithms-Fall 2025

Assignment#1

Due Date and Time: 19th September, 2025 (11:30 AM)

Total:150 marks

Instructions:

1. *Late assignment will not be accepted*
2. *Only handwritten attempt will be graded, i.e., printed attempts will not be graded*
3. *Only the attempts submitted to Mr. Fahad or Mr. Amir in the Academic office (till the due date & time) will be considered, i.e., the submissions that will be slided beneath instructors' office doors or submitted elsewhere will not be graded.*
4. *There will be no credit if the given requirements are changed*
5. *Your solution will be evaluated in comparison with the best solution (having minimum number of steps) for part 3*
6. *Please mention the question number and its part (if any) before writing down its solution. Use the same conventions used in the assignment's document.*
7. *Don't plagiarize. Plagiarism would lead to straight zero in the assignment regardless of the percentage copied.*
8. *Also submit scanned copies of your assignment on GCR. 20% penalty for not submitting it on GCR*

Question 1 [6x4=24 Marks]

Provide a worst-case asymptotic time complexity of the following algorithms by using a suitable asymptotic notation considering a nearest function. Assume that there are no errors/ bugs in the algorithms. Show the meaningful working behind your answer in the rough work column.

		Time Complexity	Rough Work
a)	<pre>Func(n) BEGIN FOR (i = 1 to n) FOR (j = 1 to i) IF (j modulus i = 1) THEN PRINT i END FOR END FOR END FOR END</pre>		

b)	<pre> for (int i = 1; i <= n*n; i = i * 3) { for (int j = 0; j < i; j++) cout << j; } </pre>		
c)	<pre> for (int i = 0; i <= n; i = i+2) { for (int j = n; j >= i; j--) { for (k = 100; k >= 1; k = k/2) cout << k; } } </pre>		
d)	<pre> for (int i = 1; i*i <= n; i++) { for (int j = n; j >= i*i; j--) { for (k = j; k >= 1; k = k/2) cout << k; for (l = 1; l >= i; l++) cout << l; } } </pre>		
e)	<pre> // Algorithm to Search in a Binary // Search Tree while x ≠ NIL and k ≠ x.key if k < x.key x = x.left else x = x.right return x </pre>		

f)	<pre> count ← 0 for i ← 1 to ⌊√n⌋ do if n mod i = 0 then count ← count + 1 if i ≠ n/i then count ← count + 1 return count </pre>		
----	--	--	--

Question 2 [4x2=8 Marks]

- Use Big-O to prove or disprove that $5n^2 + 20n \log n + 100$ is $O(n^3)$.
- Using Big-O, prove that: $2n^4 + 50n^2 \log n + 7n$ is $O(n^4)$.
- Show that $n \log n + 5n + 10$ is $O(n^2)$ but not $\Omega(n^2)$.
- Prove or disprove: $2n$ is $O(n!)$.

Question 3 [10 Marks]

You are given two arrays $A=[a_1, a_2, \dots, a_n]$ and $B=[b_1, b_2, \dots, b_m]$. Design an algorithm to check whether there exists a pair (a_i, b_j) such that the difference between them equals a given target value K , i.e.,

$$a_i - b_j = K$$

for some i, j . If such a pair exists, return the indices i, j .

Analyze the time complexity of your algorithm. Smaller time complexity and space complexity will get full marks

Question 4 [10 Marks]

Consider the following C++ code:

- Count the number of **basic operations** in each major part of the algorithm:
- Derive the **best case**, **worst case**, and **average case** time complexity of the algorithm in terms of n .
- Identify the **dominant term** in each case and give the final **Big-O complexity**.

```

int func(vector<int>& arr, int n) {

    int counter = 0;

    for (int i = 0; i < n; i++) {

        counter++;

        if (arr[i] % 3 == 0) {

            for (int j = 0; j < n; j++) {

                for (int k = 0; k < j; k++) {

                    counter++;

                }

            }

        } else if (arr[i] % 3 == 1) {

            int x = i;

            while (x > 0) {

                counter++;

                x = x / 2;

            }

        } else {

            for (int m = 1; m < n; m *= 2) {

                for (int p = 0; p < m; p++) {

                    counter++;

                }

            }

        }

    }

    for (int a = 0; a < n; a++) {

        for (int b = a; b < n; b++) {

            for (int c = b; c < n; c++) {

                counter++;

            }

        }

    }

    return counter;

}

```

```

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];
    cout << func (arr, n) << endl;
    return 0;
}

```

Question 5 [25 Marks]

You are given the following algorithm that processes an array of integers:

Algorithm RippleProcess(A):

```

n ← length(A)
threshold ← n
changed ← true

while changed do
    changed ← false

    for i ← 0 to n-2 do
        if A[i] > A[i+1] and |A[i] - A[i+1]| ≥ threshold then
            swap(A[i], A[i+1])
            changed ← true
    threshold ← max(1, threshold - 1)

    for i ← n-2 downto 0 do
        if A[i] > A[i+1] and |A[i] - A[i+1]| ≥ threshold then
            swap(A[i], A[i+1])
            changed ← true
    threshold ← max(1, threshold - 1)

```

- Analyze the algorithm and describe in your own words what it achieves. [2.5]
- Determine whether the array will always reach a specific final arrangement after execution. If yes, explain what that arrangement is. [2.5]
- Derive the **best-case**, **average-case**, and **worst-case** time complexities of the algorithm. Justify your answer. Show the dry run of all of these three cases. [15]
- Compare this algorithm's mechanism with any standard sorting algorithm you know, highlighting similarities and differences. [5]

Question 6 [8 Marks]

For each of the following pairs of functions, indicate whether the first function of each pair has a lower, same, or higher order of growth (to within a constant multiple) than the second function. Give a short justification.

- $n^2 + 50n \log n$ and $10n^2$
- $n^2 \log n$ and n^3
- $\log n$ and $\sqrt{n}$
- n^2 and $2^{\log n}$

Question 7 [10 Marks]

A recursive algorithm on input size n behaves as follows:

- It makes **two** recursive calls on subproblems of size $n/2$.
- It makes **one** additional recursive call on a subproblem of size $\sqrt{n}$
- It performs $\Theta(n)$ additional work per call to combine results.

Assume $T(1) = \Theta(1)$ and $T(n)$ is defined for integer $n \geq 2$.

- Write a recurrence for $T(n)$. [2]
- Use a recursion-tree, or iterative method to find the asymptotic growth of $T(n)$. Be explicit about how the $T(\sqrt{n})$ term influences (or does not influence) the final bound. [6]
- If the extra call were $T(n^\alpha)$ for some fixed $0 < \alpha < 1$, for which α (if any) would the overall order change from $\Theta(n \log n)$? Briefly justify [2]

Question 8 [4*5=20 Marks]

Write the **recurrence relation**, solve it, and clearly state the final asymptotic complexity.

Code Snippet 1 – fun1	Code Snippet 2 – fun2
<pre>int fun1(int n) { if (n <= 1) return 1; for (int i = 0; i < n; i++) { /* O(1) work */ } return fun1(n/2) + fun1(n/2); }</pre>	<pre>void fun2(int n) { if (n == 0) return; for (int i = 0; i < n; i++) { fun2(n - 1); } }</pre>

Code Snippet 3 – fun3 <pre> int fun3(int n) { if (n <= 2) return 1; for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { /* O(1) work */ } } return fun3(n/2) + fun3(n/3); } </pre>	Code Snippet 4 – fun4 <pre> void fun4(int n) { if (n <= 1) return; for (int i = 0; i < n; i++) { /* O(1) work */ } fun4(n-1); fun4(n-1); } </pre>
---	---

For each function above (fun1, fun2, fun3, fun4):

1. Write the recurrence relation.
2. Solve it using a **recursion tree or iterative method**
3. Clearly state the **asymptotic time complexity** (Big- Θ).

Question 9 [5x3=15 Marks]

Solve the following recurrence relations using iteration method.

- a. $T(n) = T(n-1) + n^2$, $T_1 = 2$
- b. $T(n) = 2T(n/2) + 3$, $T_1 = 1$
- c. $T(n) = T(n-1) + 1/n$, $T_1 = 2$

Question 10 [4*5=20 Marks]

Solve the following recurrence relations using recursion tree method. Also, prove your answer using iteration method.

- a. $T(n) = T(n-1) + T(n/2) + n$, $T(1) = 1$.
- b. $T(n) = 4T(n/2) + n \log n$, $T(1) = 1$
- c. $T(n) = 9T(n/3) + n^2 \log n$, $T(1) = 1$
- d. $T(n) = T(n-1) + 2^n$, $T(0) = 1$